Machine Learning Operations (AIMLCZG523)

Assignment 01

Heart Disease Risk Prediction: an End-to-End MLOps Pipeline

Data acquisition · EDA · model development · MLflow tracking · packaging · CI/CD · Docker · Kubernetes · monitoring

<Student name> | <BITS ID>

Code repository: https://github.com/<your-username>/heart-disease-mlops

1. Problem, objective and summary

The goal is to predict whether a patient has heart disease from 13 routinely collected clinical attributes and to expose the model as a reproducible, containerised, monitored REST API. The dataset is the Cleveland subset of the UCI Heart Disease database (303 patients). The original target num (0-4) is binarised: 0 = no disease, 1-4 = disease present.

Outcome	Value
Champion model	Logistic regression (L2, C = 0.3, class_weight = balanced)
Selection metric (5-fold CV ROC-AUC, train split)	0.907
Held-out test ROC-AUC / accuracy	0.961 / 0.869
Held-out test recall / precision	0.929 / 0.812
Runs tracked in MLflow	68 (64 nested tuning runs)
Unit tests	28 pytest tests, flake8 clean
Serving	FastAPI in Docker, 2-5 replicas on Kubernetes behind a LoadBalancer + Ingress, Prometheus/Grafana monitoring

1.1 Architecture

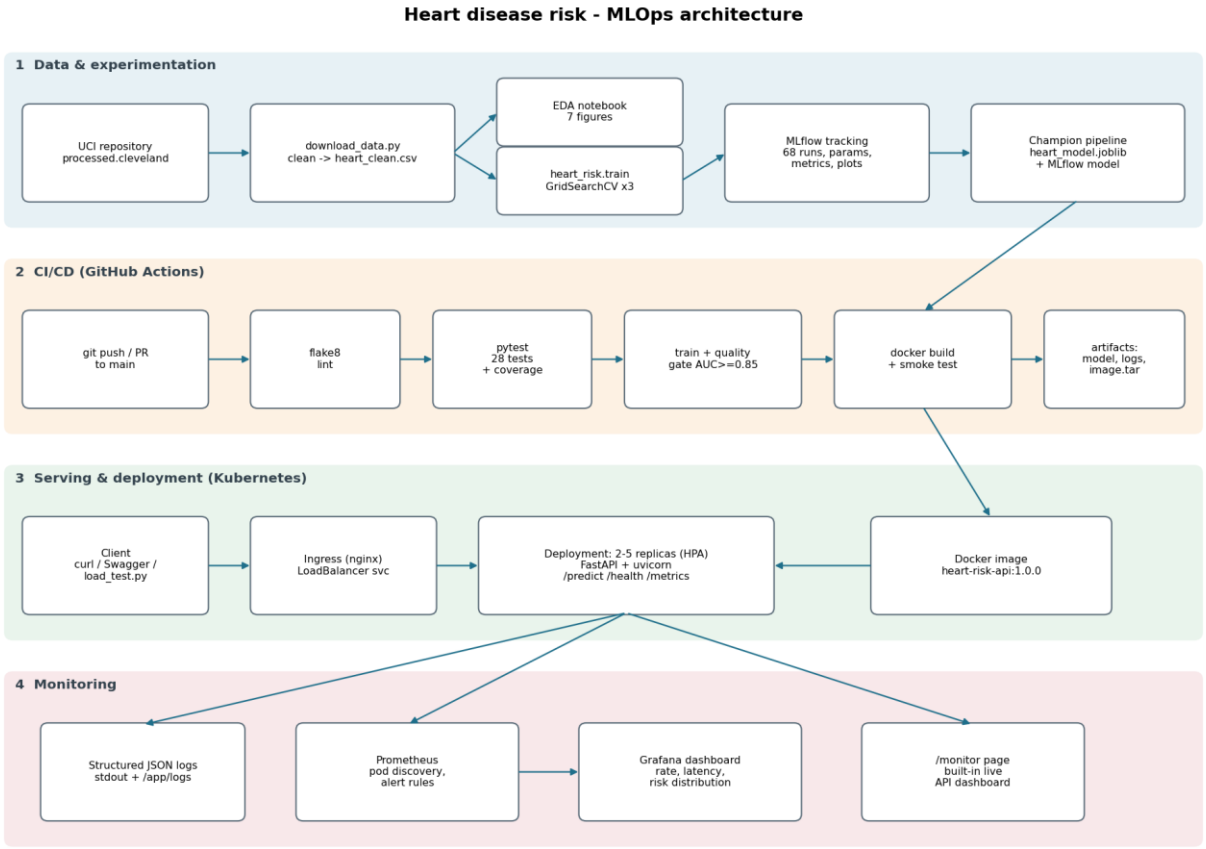


Figure 1 - System architecture: data & experimentation, CI/CD, serving on Kubernetes and monitoring.

Design principle: every stage is a plain Python entry point (scripts/download_data.py, python -m heart_risk.train, uvicorn api.main:app) so the same commands run on a laptop, in GitHub Actions and inside the container. All preprocessing lives inside a single scikit-learn Pipeline object, which is the artefact that is versioned, containerised and served - this removes train/serve skew.

2. Setup and installation

Requirements: Python 3.11, Git; Docker and Minikube (or Docker Desktop Kubernetes) for the deployment steps. All library versions are pinned in requirements.txt; requirements-api.txt is the minimal runtime subset used by the container (same pins, so the pickled pipeline always loads).

```
git clone <repo-url> && cd heart-disease-mlops
```

```
python -m venv .venv && .venv\Scripts\activate      # Linux: source .venv/bin/activate
pip install -r requirements-dev.txt
python scripts/download_data.py      # download + clean the UCI data
python -m heart_risk.eda             # EDA figures
python -m heart_risk.train           # tune 3 models, log to MLflow, save champion
pytest -v && flake8                  # tests + lint
uvicorn api.main:app --port 8000     # serve locally -> http://127.0.0.1:8000/docs
```

Folder / file	Purpose
heart_risk/	Package: config, data cleaning, features, training, evaluation, inference, EDA
api/	FastAPI app, request/response schemas, logging + metrics, /monitor page
notebooks/	01_eda, 02_model_training, 03_inference (executed)
data/raw, data/processed	Raw UCI file and cleaned CSV
models/	heart_model.joblib, MLflow model, model_metadata.json
mlruns/	MLflow tracking store (all runs, metrics, artefacts)
test/	pytest unit tests (data, features, model, API)
.github/workflows/ci.yml	Lint -> test -> train -> Docker pipeline
Dockerfile, docker-compose.yml	Serving image; local API + Prometheus + Grafana stack
k8s/	Namespace, Deployment, Service, Ingress, HPA, monitoring
monitoring/	Prometheus config, alert rules, Grafana dashboard
screenshots/, reports/	Evidence and this report

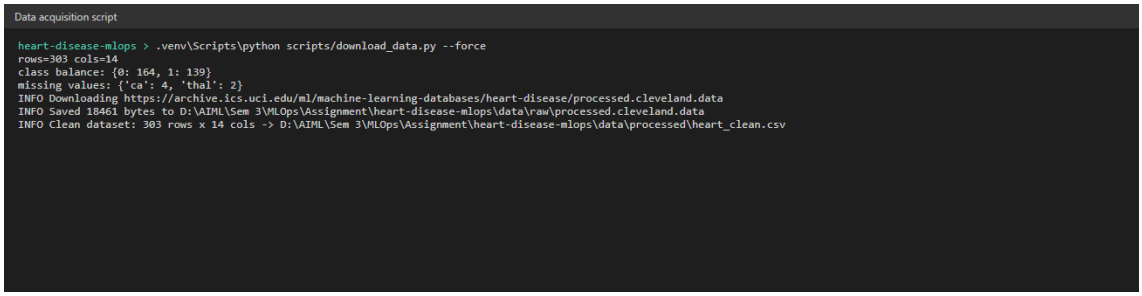


Figure 2 - Data acquisition script downloading and cleaning the data.

3. Data acquisition, cleaning and EDA

3.1 Cleaning

- **Missing values:** the raw file marks missing values with '?'. Only ca (4 rows) and thal (2 rows) are affected. They are kept as NaN in the cleaned CSV and imputed inside the pipeline (median for numeric, most-frequent for categorical). Imputing in the pipeline means the statistics are learned on training folds only and the identical rule is applied to API requests - the API accepts requests with ca/thal omitted.
- **Validity rules:** values outside physiological ranges (e.g. chol < 100) or invalid category codes are converted to NaN; none were found in this subset but the rule protects future data.
- **Types and duplicates:** all columns cast to numeric, binary columns to int64 (fixed width so the MLflow signature is platform independent), duplicate rows dropped (none).

3.2 Findings

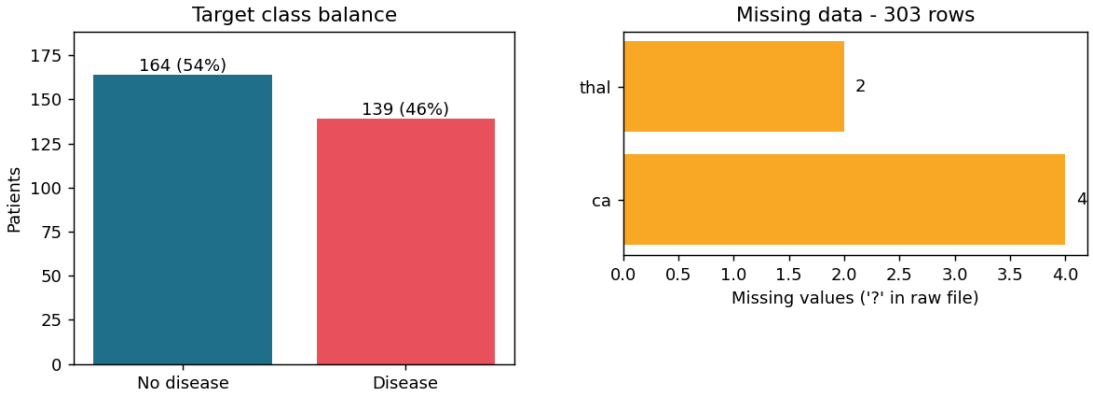


Figure 3 - Class balance (164 healthy vs 139 diseased, 54 / 46 %) and missing values in the raw file.

The classes are only mildly imbalanced, so stratified splits are sufficient; class_weight='balanced' is still explored during tuning.

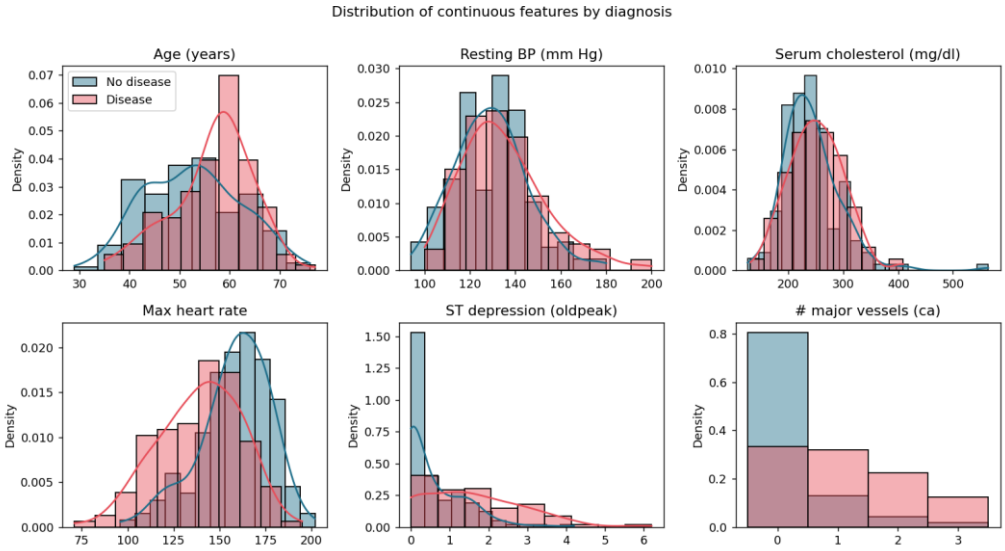


Figure 4 - Continuous features by diagnosis.

- Diseased patients reach a markedly lower maximum heart rate (thalach) and show higher exercise-induced ST depression (oldpeak).
- Cholesterol and resting blood pressure overlap strongly between classes - weak individual predictors.
- The number of coloured vessels (ca) shows a monotonic risk increase, so it is treated as an ordinal numeric feature.

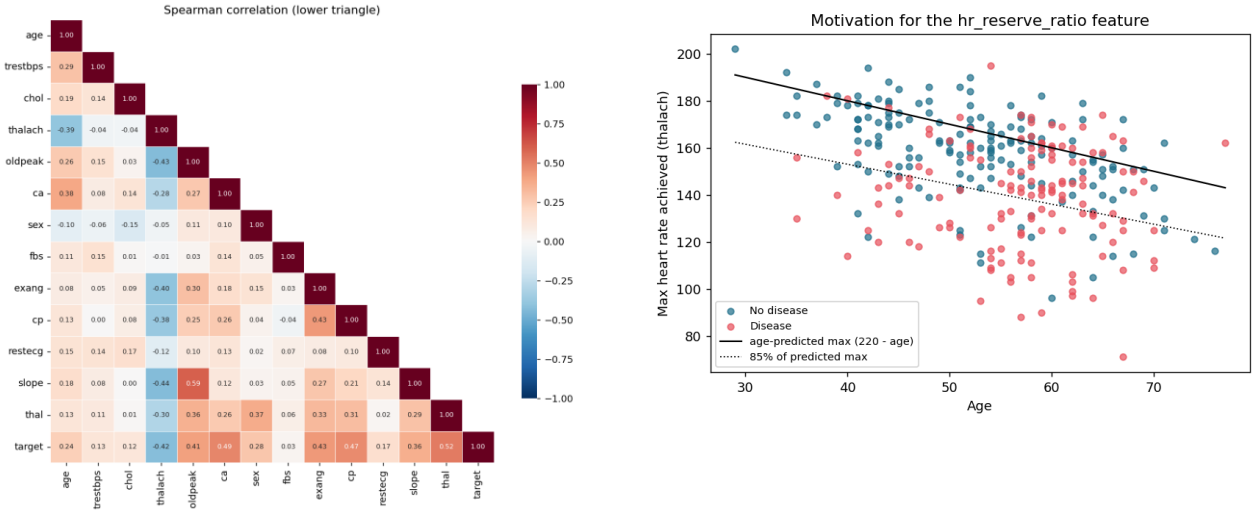


Figure 5 - Spearman correlation heatmap (left) and max heart rate vs age (right): most diseased patients fall below 85 % of the age-predicted maximum.

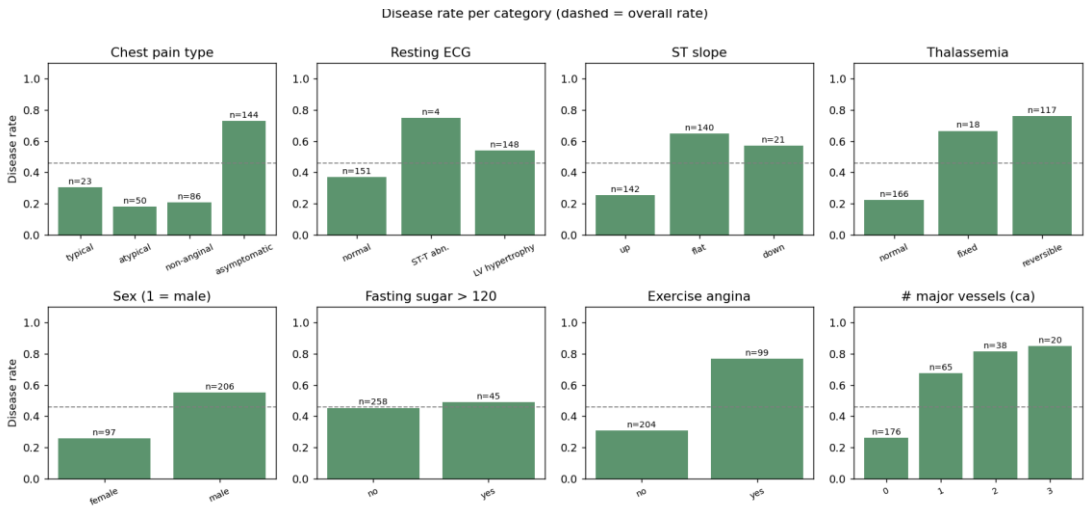


Figure 6 - Disease rate per category (dashed line = overall rate).

Chest pain type 4 (asymptomatic), reversible thalassemia defect, flat ST slope and exercise-induced angina all have disease rates above 65 %. Because these are nominal codes (cp = 4 is not 'four times' cp = 1), cp, restecg, slope and thal are one-hot encoded.

Figure 5 (right) motivates the engineered feature hr_reserve_ratio = thalach / (220 - age): the fraction of the age-predicted maximum heart rate reached during the stress test. It separates the age effect from the cardiac response. Box plots (reports/figures/eda/06_boxplots.png) show a few high-cholesterol outliers that are physiologically plausible and were therefore kept.

4. Feature engineering and model development

4.1 Preprocessing pipeline

Step	Columns	Transformation
ClinicalFeatureBuilder	age, thalach	adds hr_reserve_ratio
numeric	age, trestbps, chol, thalach, oldpeak, ca, hr_reserve_ratio	median impute -> StandardScaler
binary	sex, fbs, exang	most-frequent impute (kept 0/1)
categorical	cp, restecg, slope, thal	most-frequent impute -> one-hot with fixed UCI category levels, handle_unknown='ignore'

13 raw inputs become 23 model features. Fixing the category levels from the UCI documentation guarantees a stable feature layout even if a level is missing from a training fold or a request.

4.2 Models, tuning and selection

- **Split:** 80 / 20 stratified train/test split (242 / 61 patients, random_state = 42). The test split is never used for any decision.
- **Cross-validation:** StratifiedKFold(5, shuffle) on the training split; metrics accuracy, precision, recall, F1 and ROC-AUC.
- **Logistic regression:** C in {0.01 ... 3}, penalty L1/L2, class_weight None/balanced (24 configurations).
- **Random forest:** 300 trees, max_depth {3, 5, 8, None}, min_samples_leaf {1, 4, 8}, max_features {sqrt, 0.5} (24).
- **Gradient boosting:** n_estimators {100, 200}, learning_rate {0.03, 0.1}, max_depth {2, 3}, subsample {0.7, 1.0} (16).
- **Selection rule:** GridSearchCV refits on ROC-AUC (a threshold-free ranking metric suited to a risk score); the champion is the family with the highest mean CV ROC-AUC.

Model	CV AUC	CV acc	CV recall	Test AUC	Test acc	Test prec	Test recall
logistic regression	0.907	0.855	0.801	0.961	0.869	0.812	0.929
random forest	0.900	0.818	0.764	0.959	0.885	0.839	0.929
gradient boosting	0.886	0.814	0.764	0.961	0.902	0.867	0.929

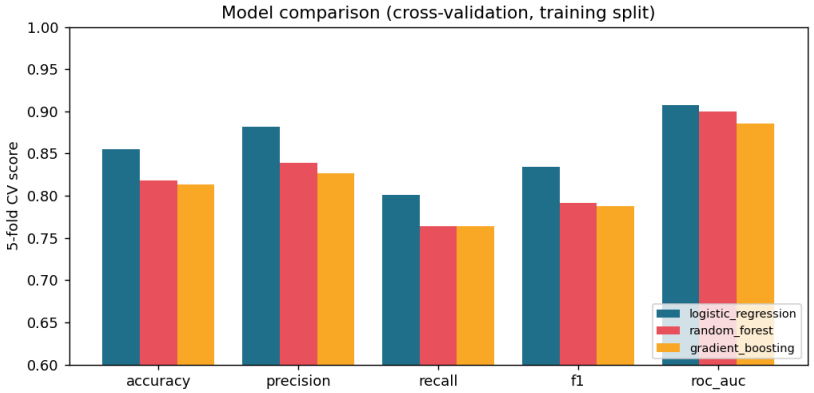


Figure 7 - Cross-validated metrics of the tuned model families.

Logistic regression wins on CV ROC-AUC (0.907) and accuracy. The tree ensembles score slightly higher on the 61-patient test split, but that difference (+/- 1-2 patients) is within noise and choosing on it would overfit the test set. With 242 training rows and a largely additive signal, the simpler, interpretable linear model is the better production choice: it is fast, small, well calibrated and its coefficients can be explained to clinicians.

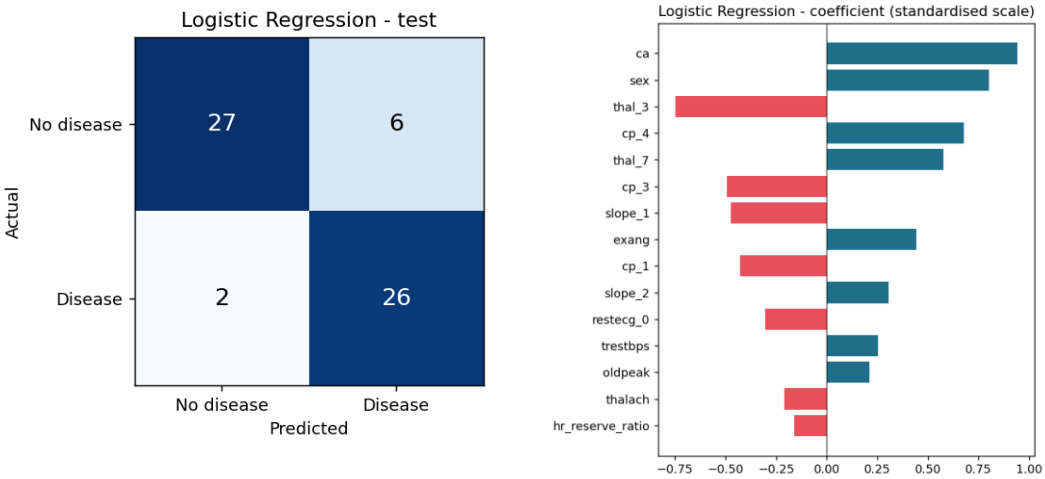


Figure 8 - Champion confusion matrix on the test split (left) and coefficients on standardised features (right).

Recall of 0.93 on the test split (26 of 28 diseased patients found) is the clinically important number: a screening model should prefer false alarms over missed cases. The most influential features - ca, sex, thal, cp = 4 and exang - match the EDA. thalach and hr_reserve_ratio are correlated and share their weight.

5. Experiment tracking (MLflow)

- **Parent run per model family:** best hyper-parameters, CV metrics, test metrics, tags (model_family, stage, package_version), plots (confusion matrix, ROC, PR, feature importance), the full CV results CSV and the logged sklearn model with input signature and example.
- **Nested child run per grid point:** 64 runs with parameters and mean / std of all five CV metrics - every configuration tried is traceable.
- **champion-selection run:** tags the winning family and run id and logs the comparison chart, comparison CSV and model_metadata.json.

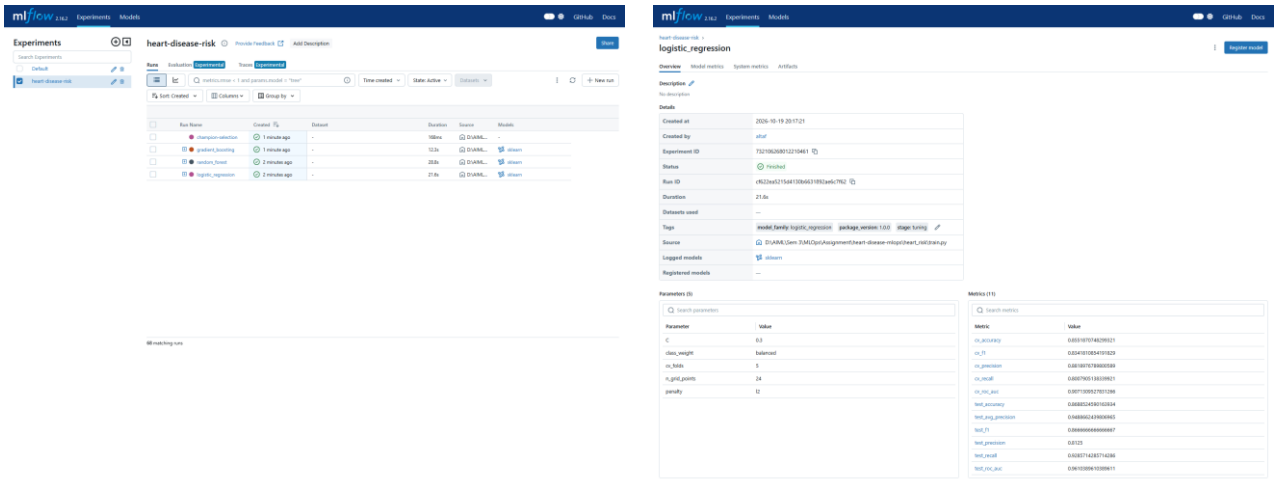


Figure 9 - MLflow experiment 'heart-disease-risk' (left) and the champion run with parameters and metrics (right).

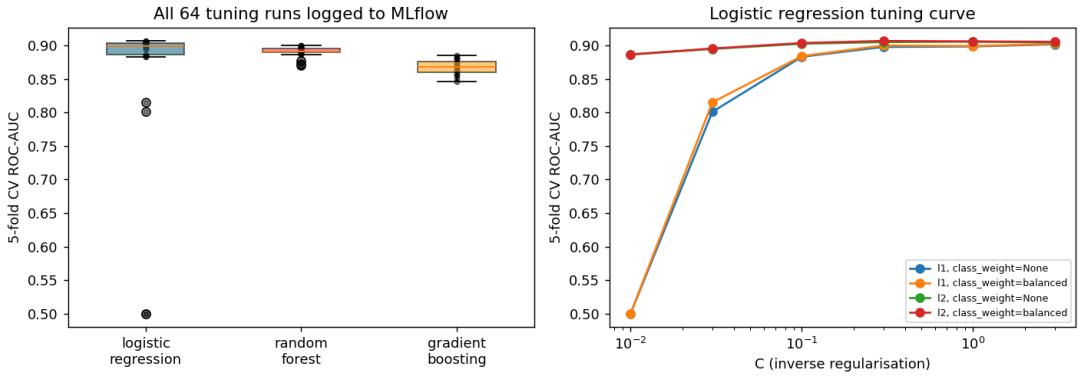


Figure 10 - All 64 tuning runs read back from MLflow (scripts/experiment_summary.py).

The tracking data also explains the tuning behaviour: strong L1 regularisation (C <= 0.03) removes every coefficient (AUC 0.5), and logistic regression plateaus at ~0.90-0.91 from C = 0.1. Random-forest configurations are tightly clustered (0.87-0.90), while gradient boosting with deeper trees overfits.

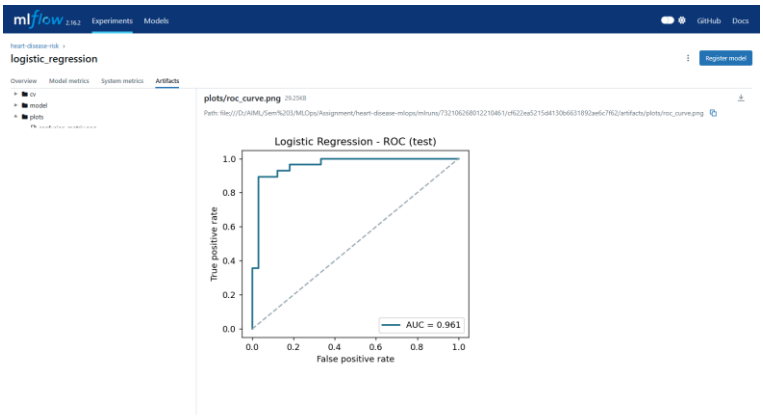


Figure 11 - Logged artefacts of a run (plots, cv results, model).

6. Model packaging and reproducibility

- **models/heart_model.joblib:** the complete Pipeline (feature builder + imputers + scaler + encoder + classifier). Raw JSON fields in, probability out - used by the API.
- **models/mlflow_model/:** the same pipeline in MLflow format (MLmodel, conda.yaml, python_env.yaml, requirements.txt, signature, input example) - loadable with mlflow.pyfunc.load_model.
- **models/model_metadata.json:** version, family, MLflow run id, best params, metrics, feature list, decision threshold, training time.
- **Determinism:** fixed random_state for split, CV and models; re-training from scratch reproduced identical metrics.

- **Environment:** pinned requirements.txt (+ requirements-api.txt for serving); Python 3.11 locally, in CI and in the image.

7. CI/CD pipeline and automated testing

Test module	What is verified
test_data.py (7)	binary target mapping, '?' parsing, missing values kept for imputation, out-of-range and invalid codes -> NaN, de-duplication, download skipped when cached
test_features.py (5)	hr_reserve_ratio value, no NaNs after preprocessing, stable 23-column output even with missing categories, scaling, feature names
test_model.py (5)	models learn signal (AUC > 0.75), metric values, prediction contract, save/load determinism, plots written
test_api.py (11)	/health, /predict output + request id header, optional fields, 422 on invalid / unknown fields, batch, /metrics, /monitor, /model-info

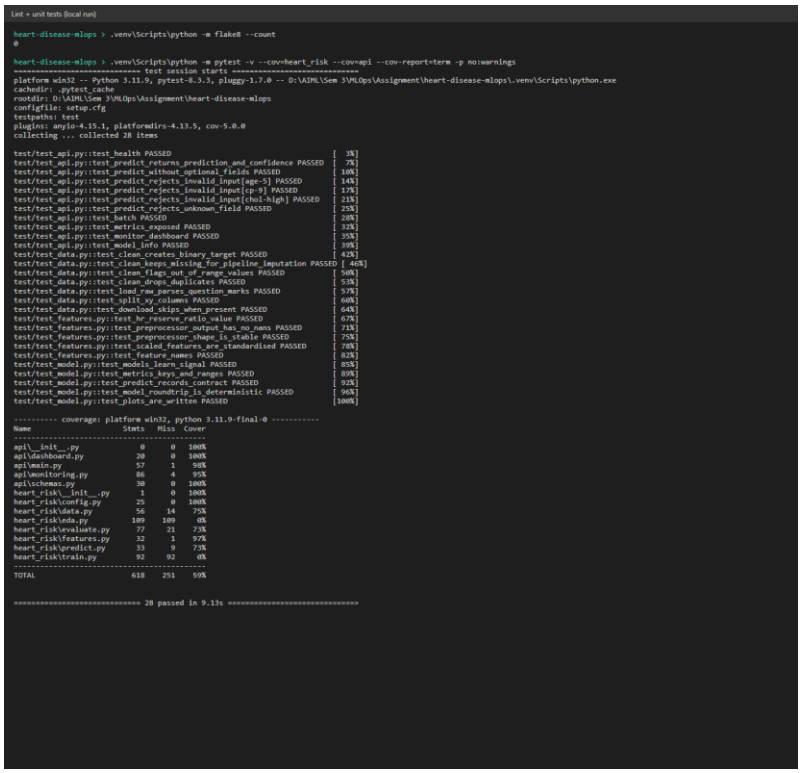


Figure 12 -flake8 (0 issues) and 28 passing tests with coverage.

GitHub Actions workflow (.github/workflows/ci.yml), triggered on push, pull request and manually:

Job	Steps	Artefacts
lint	flake8 over all code	flake8-report
test (needs lint)	pytest -v, JUnit XML, coverage	test-reports
train (needs test)	download data, full training with MLflow, quality gate: test ROC-AUC >= 0.85	trained-model, mlflow-runs, training.log
docker (needs train)	build image with the freshly trained model, run it, wait for /health, POST sample_request.json, assert response	docker-smoke-test, container.log, docker-image

Jobs are chained with needs:, so any lint error, failing test, training crash, quality-gate miss or broken container stops the pipeline, and each failing step keeps its full log in the run.

[Insert screenshot: screenshots/07_github_actions_pipeline.png]

Figure 13 - GitHub Actions run with all jobs and artefacts.

8. Model containerisation and API

- **Image:** multi-stage build on python:3.11-slim (wheels built in a builder stage), runs as non-root uid 1001, HEALTHCHECK on /health, only heart_risk/, api/ and the model files are copied.
- **POST /predict:** validated JSON (Pydantic: ranges, allowed codes, unknown fields rejected) -> prediction, label, confidence (probability of the predicted class), risk_probability, model_version, request_id.
- **Other endpoints:** /predict/batch (up to 500 patients), /health, /model-info, /metrics (Prometheus), /monitor (live dashboard), /docs.

```
docker build -t heart-risk-api:1.0.0 .
docker run -d --name heart-api -p 8000:8000 heart-risk-api:1.0.0
curl -X POST http://localhost:8000/predict -H "Content-Type: application/json" -d
@sample_request.json
{"prediction":1,"label":"heart_disease","confidence":0.9767,"risk_probability":0.9767,
 "model_version":"1.0.0","request_id":"adccdac868c6"}
```

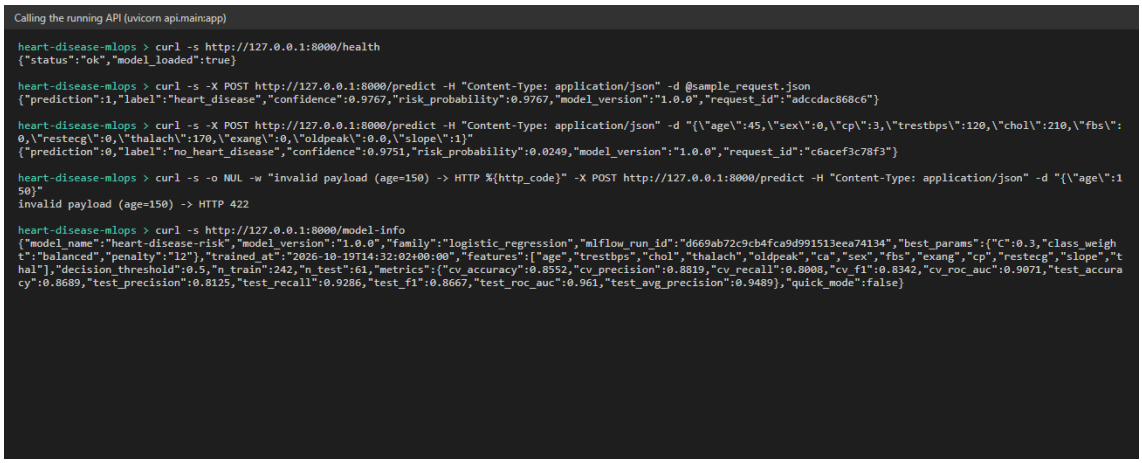


Figure 14 - Health check, predictions for a high- and a low-risk patient, validation error (422) and model info.

[Insert screenshot: screenshots/14_docker_build.png]

[Insert screenshot: screenshots/15_docker_run_predict.png]

Figure 15 - Docker image build (left) and the container answering /predict (right).

9. Production deployment (Kubernetes)

Manifest	Key settings
00-namespace.yaml	isolated namespace heart-risk
01-deployment.yaml	2 replicas, RollingUpdate (maxUnavailable 0), readiness + liveness probes on /health, CPU / memory requests and limits, non-root, Prometheus scrape annotations
02-service.yaml	type LoadBalancer, port 80 -> 8000
03-ingress.yaml	nginx Ingress, host heart-risk.local
04-hpa.yaml	autoscale 2-5 replicas at 70 % CPU
monitoring/*.yaml	Prometheus (RBAC + pod discovery) and Grafana with provisioned datasource and dashboard

```
minikube start
bash scripts/deploy_k8s.sh        # build, load image, apply manifests + monitoring
minikube tunnel                   # separate terminal
kubectl -n heart-risk get pods,svc,ingress,hpa
curl -X POST http://<EXTERNAL-IP>/predict -H "Content-Type: application/json" -d @sample_request.json
```

Access instructions: the API is deployed on a local Minikube cluster. With minikube tunnel running it is reachable at http://<EXTERNAL-IP>/ (LoadBalancer) or http://heart-risk.local/ after adding the Minikube IP to the hosts file (Ingress). Docker Desktop Kubernetes publishes the LoadBalancer on http://localhost/.

[Insert screenshot: screenshots/16_k8s_resources.png]

[Insert screenshot: screenshots/17_k8s_predict_loadbalancer.png]

Figure 16 - Pods, Service, Ingress and HPA in namespace heart-risk (left) and a prediction served through the LoadBalancer (right).

10. Monitoring and logging

- **Request logging:** middleware assigns a request id (returned in x-request-id), times each request and writes one JSON line per request and per prediction (inputs, label, probability) to stdout and /app/logs/api.log - ready for kubectl logs or a log shipper.
- **Prometheus metrics:** heart_api_requests_total{endpoint,status}, heart_api_request_latency_seconds (histogram), heart_api_predictions_total{label}, heart_api_risk_probability (histogram), heart_api_inference_errors_total, heart_api_model_info.
- **Alert rules:** API down, 5xx rate > 5 %, p95 /predict latency > 500 ms, and a simple drift signal - positive-prediction share above 80 % for 15 min (training prevalence is 46 %).
- **Dashboards:** Grafana dashboard (status, totals, positive share, error rate, latency percentiles, request rate, predictions per label, risk distribution) and a dependency-free /monitor page built into the API.

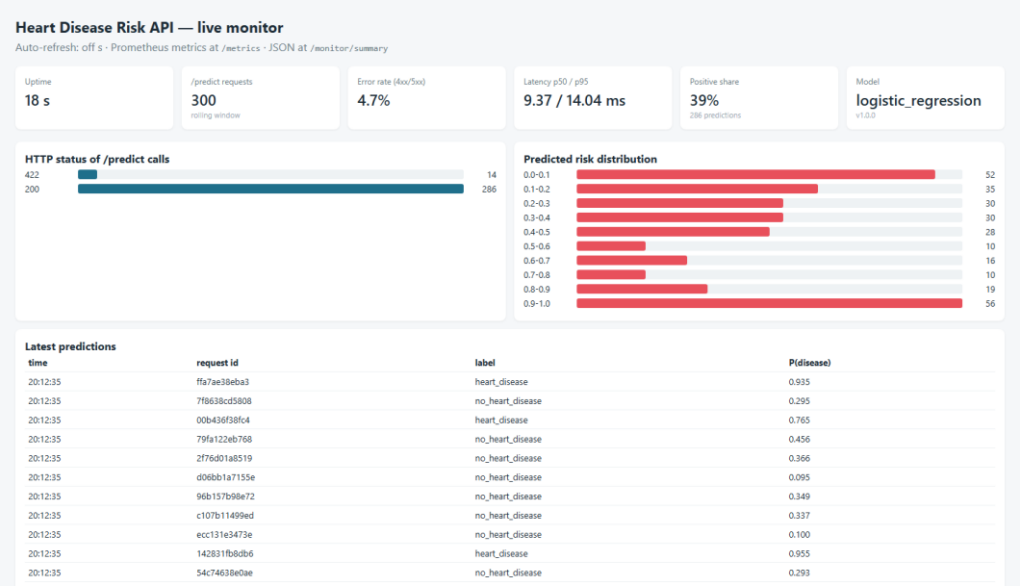


Figure 17 - Built-in /monitor dashboard after 300 requests from scripts/load_test.py (5 % deliberately invalid).



Figure 18 - Structured JSON request and prediction logs.

[Insert screenshot:

[Insert screenshot:

screenshots/18_grafana_dashboard.png] screenshots/19_prometheus_targets.png]

Figure 19 - Grafana dashboard fed by Prometheus (left) and Prometheus scraping every API pod (right).

11. Conclusion and future work

The project delivers a reproducible path from raw UCI data to a monitored API: one pipeline object carries all preprocessing, every experiment is tracked, CI re-trains and gates the model before building the image, and Kubernetes runs it with health checks, autoscaling and metrics. Next steps: a model registry with staged promotion, pushing the image to a container registry from CI, decision-threshold tuning with clinicians, and statistical input-drift tests on logged requests.

Code repository: **https://github.com/<your-username>/heart-disease-mlops**